

Mantle: mStocks

Security Review

Cantina Managed review by:
0xRajeev, Lead Security Researcher
Jiri123, Associate Security Researcher

July 15, 2026

Contents

1 Introduction **2**
1.1 About Cantina . . . 2
1.2 Disclaimer . . . 2
1.3 Risk assessment . . . 2
1.3.1 Severity Classification . . . 2

2 Security Review Summary **3**
2.1 Scope . . . 3

3 Findings **4**
3.1 Low Risk . . . 4
3.1.1 `TokenIssuer` proxy deployment has no upgrade path . . . 4
3.1.2 `TokenIssuer` emergency sweep is blocked while paused . . . 4
3.1.3 `updateSymbol` bypasses factory symbol validation . . . 4
3.1.4 Token symbol and asset code can diverge from the factory registry . . . 5
3.1.5 Missing empty-array check for `newList` in `replaceSanctions` allows wiping all sanctions without replacing . . . 5
3.1.6 `assetCode` missing format or uniqueness validation allows invalid/duplicate tokens 6
3.1.7 `TokenIssuer.__gap` with incorrectly declared slots may cause issues during upgrade 7
3.1.8 Missing overwrite/acceleration protection on scheduled UI multiplier changes is risky 7
3.2 Informational . . . 7
3.2.1 Incorrect comment states `ReentrancyGuard` uses transient storage . . . 7
3.2.2 Unused `Executed` event remains in `TokenIssuer` . . . 8
3.2.3 ERC-8056 is a Draft Standard that needs to be monitored for upcoming changes . . . 8
3.2.4 `DEFAULT_ADMIN_ROLE` lacks transfer safeguards . . . 9
3.2.5 `receive()` is unnecessary in `TokenIssuer` . . . 9
3.2.6 Missing zero-address check on `admin_` in `ScaledToken.__scaledToken_init` . . . 10
3.2.7 `SymbolMismatch` revert passes a description string as the provided value, not actual data . . . 10
3.2.8 Centralization risk from privileged roles and flows . . . 10

DRAFT

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

DRAFT

2 Security Review Summary

From Jul 6th to Jul 10th the Cantina team conducted a review of mStocks-protocol on commit hash f09a8e49. The team identified a total of **16** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	8	8	0
Gas Optimizations	0	0	0
Informational	8	5	3
Total	**16**	**13**	**3**

2.1 Scope

The security review had the following components in scope for mStocks-protocol on commit hash f09a8e49:

```
src
├── complianceRegistry
│   ├── ComplianceGuardUpgradeable.sol
│   ├── ComplianceRegistry.sol
│   └── IComplianceRegistry.sol
├── ERC8056
│   ├── ERC8056BaseUpgradeable.sol
│   ├── IERC8056Scheduled.sol
│   ├── IScaledUIAmount.sol
│   ├── IScaledUIAmountBalances.sol
│   ├── IScaledUIAmountConversion.sol
│   └── IScaledUIAmountNewUIMultiplier.sol
├── pauseRegistry
│   ├── IPauseRegistry.sol
│   ├── PauseGuardUpgradeable.sol
│   └── PauseRegistry.sol
├── scaledToken
│   ├── ScaledToken.sol
│   └── ScaledTokenFactory.sol
└── tokenIssuer
    ├── IMintAndBurnable.sol
    └── TokenIssuer.sol
```

DRAFT

3 Findings

3.1 Low Risk

3.1.1 `TokenIssuer` proxy deployment has no upgrade path

Severity: Low Risk.

Context: TokenIssuer.sol#L22.

Description: `TokenIssuer` is deployed behind an `ERC1967Proxy`, but the implementation does not expose an upgrade path. It does not inherit UUPS upgrade logic or define `_authorizeUpgrade`, and `ERC1967Proxy` has no external upgrade function of its own. As a result, the implementation cannot be changed after deployment despite the proxy-based deployment pattern.

Recommendation: Use an upgradeable proxy pattern with an external admin path, such as `TransparentUpgradeableProxy` with `ProxyAdmin`, or deploy `TokenIssuer` directly if it is intended to be immutable.

Mantle: Fixed in PR 3.

Cantina Managed: Fix verified.

3.1.2 `TokenIssuer` emergency sweep is blocked while paused

Severity: Low Risk.

Context: TokenIssuer.sol#L117.

Description: `TokenIssuer.withdraw()` is an admin-only emergency sweep function, but it is gated by `whenNotPaused`. If the issuer is paused during incident response, the admin cannot sweep custody assets without first unpausing or performing extra role-revocation steps. This makes the emergency recovery path harder to use during the state where it is most likely needed.

Recommendation: Remove `whenNotPaused` from `withdraw()` so the admin can sweep assets while the issuer is paused.

Mantle: Fixed in PR 3.

Cantina Managed: Fix verified.

3.1.3 `updateSymbol` bypasses factory symbol validation

Severity: Low Risk.

Context: ScaledToken.sol#L139-L143.

Description: `ScaledToken.updateSymbol()` only checks that the new symbol is non-empty. It does not apply the factory symbol rules for charset and length. As a result, an admin can set a live token symbol that would be rejected during token creation or factory rename.

Recommendation: Apply the same symbol validation rules used by the factory inside `updateSymbol()` before emitting `SymbolUpdated` and updating storage.

Mantle: Fixed in PR 12.

Cantina Managed: Fix verified.

DRAFT

3.1.4 Token symbol and asset code can diverge from the factory registry

Severity: Low Risk.

Context: ScaledToken.sol#L139-L143.

Description: `ScaledToken.updateSymbol()` updates the token symbol directly, but the factory `tokenBySymbol` registry is not updated in the same call. This can leave `token.symbol()` and the factory registry out of sync, creating stale or conflicting symbol mappings for live tokens.

The same issue is also present for token asset code.

Recommendation: Consider requiring symbol changes to be synchronized through the factory registry, or make the factory rename flow verify the token's current symbol before updating `tokenBySymbol` while enforcing that it is always called. Consider similar changes for token asset code.

Mantle: Fixed in:

- PR 7.
- PR 10.

Cantina Managed: PR 7 fixes `renameSymbol()` to verify the token's current symbol before updating `tokenBySymbol`. However, this assumes that `updateSymbol()` is always followed by a call to `renameSymbol()`. If not called, the factory registry for that token will have a stale symbol and will be out of sync with `ScaledToken` value. The same issue applies to token asset code.

The same issue is present in PR 10.

Mantle: Acknowledged. This is the same root cause already discussed under asset code: `updateSymbol()` / `updateAssetCode()` don't enforce factory-level uniqueness or sync the factory mapping, so the admin must call `renameSymbol()` / `renameAssetCode()` afterwards to keep them in sync. We acknowledge and will keep the current design, with uniqueness enforced operationally (off-chain uniqueness check before a rename, serialized execution, and monitoring that reconciles on-chain values against the factory mapping).

Cantina Managed: Acknowledged.

3.1.5 Missing empty-array check for `newList` in `replaceSanctions` allows wiping all sanctions without replacing

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `sanctionUsers` and `unsanctionUsers` both revert with `EmptyAddressList()` when called with an empty array. However, `replaceSanctions` has no such check — passing an empty `newList` silently bumps the epoch and clears the entire sanctions list with no revert, functionally duplicating `clearSanctions`.

Impact: A caller who intends to replace the list but accidentally passes an empty array (e.g. due to an offchain bug or malformed input) would unknowingly wipe all sanctions instead of getting an error for the missing replacement list.

Recommendation: Consider adding the same `if(newList.length==0)revertEmptyAddressList();` guard used in `sanctionUsers` / `unsanctionUsers` for consistency and defensive programming.

Mantle: Fixed in PR 11.

DRAFT

Cantina Managed: Fix verified.

3.1.6 `assetCode` missing format or uniqueness validation allows invalid/duplicate tokens

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `assetCode` is documented as an "external instrument code such as ISIN, CUSIP, or internal reference." For a regulated tokenized-security protocol, this is meant to be the onchain anchor tying a `ScaledToken` instance to a specific real-world instrument. Real ISINs are a fixed 12-character alphanumeric format and CUSIPs are 9, but `updateAssetCode` only checks for a non-empty string, same as `initialize` check.

However, nothing enforces length, character set, or a checksum, and nothing prevents two separate `ScaledToken` deployments from being assigned the identical `assetCode`, since each token stores its own independent copy with no cross-token validation. Two tokens could end up both claiming to represent the same ISIN with no onchain signal that anything is wrong.

Impact: An admin could mistakenly deploy or update a token with invalid asset code or with one that collides with another instrument's `assetCode`, causing integrators or investors to be misled into treating the wrong token as backing a given real-world security.

Recommendation: Consider:

1. Format-validating `assetCode` as required for ISIN/CUSIP at both `initialize` and `updateAssetCode`.
2. Introducing a factory-level uniqueness check (similar to `tokenBySymbol` logic) across deployed `ScaledToken` instances so a duplicate `assetCode` cannot be assigned silently.

Mantle: Fixed in PR 10.

Cantina Managed: PR 10 adds format validation and factory-level uniqueness check for `assetCode`. However, `ScaledToken.updateAssetCode()` does not enforce a similar uniqueness check as performed by `ScaledTokenFactory.renameAssetCode()`, which means that `updateAssetCode()` allows duplicating a token asset code but `renameAssetCode()` will revert when called later for renaming. The same issue is present for `updateSymbol()` and `renameSymbol()`.

Mantle: Thanks for the confirmation. We agree the finding is valid — `updateAssetCode()` (and `updateSymbol()`) do not enforce the factory-level uniqueness check, so calling them directly can duplicate another token's code and leave tokenByAssetCode stale. After review, we have decided to keep the current design and not change the contracts, for the following reasons:

- Renaming an asset code/symbol is a rare, privileged governance action performed by a controlled multisig admin, not a routine flow.
- The intended procedure is two steps: first call `updateAssetCode()` on the token, then call `renameAssetCode()` on the factory to sync the mapping. The drift and uniqueness checks in `renameAssetCode()` exist precisely for this purpose.
- Uniqueness will be enforced operationally: an off-chain uniqueness check of the target code before calling `updateAssetCode()`, serialized execution during a rename (no concurrent `createToken()` / other renames), and monitoring that reconciles each token's on-chain `assetCode()` against the factory mapping.

Cantina Managed: Acknowledged.

DRAFT

3.1.7 `TokenIssuer.__gap` with incorrectly declared slots may cause issues during upgrade

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `TokenIssuer` declares 2 storage slots (`recipientIndex` mapping, `approvedRecipients` array) followed by `uint256[47]private__gap` , totaling 49 reserved slots. Every other gapped contract in this codebase follows the 50-slot-per-level convention, e.g. `ScaledToken` (4 + 46 = 50), `PauseGuardUpgradeable` (1 + 49 = 50), `ComplianceGuardUpgradeable` (1 + 49 = 50), `ERC8056BaseUpgradeable` (3 + 47 = 50). Only `TokenIssuer` is inconsistent at 49.

No storage collision exists today. However, the risk is deferred: if a future upgrade adds state variables to `TokenIssuer` and assumes the project's standard 50-slot budget rather than counting this contract's actual `__gap` size, it could shrink `__gap` incorrectly and cause a storage collision with whatever follows it in the proxy layout.

Recommendation: Consider changing `uint256[47]private__gap;` to `uint256[48]private__gap;` to align with the codebase's 50-slot-per level convention.

Mantle: Fixed in PR 3.

Cantina Managed: Fix verified.

3.1.8 Missing overwrite/acceleration protection on scheduled UI multiplier changes is risky

Severity: Low Risk.

Context: *(No context files were provided by the reviewer)*.

Description: `ERC8056BaseUpgradeable._setUIMultiplier` allows any pending scheduled change to be overwritten by a new call, with only `effectiveAtTimestamp>block.timestamp` enforced. The base contract's own NatSpec flags this as a security consideration, recommending implementers ”override `_beforeMultiplierUpdate` to prevent overwrites or acceleration”.

`ScaledToken` overrides `_setUIMultiplier` only to cap the scheduling horizon at 365 days but does not override `_beforeMultiplierUpdate` , leaving the default no-op hook in place. As a result, `MULTIPLIER_SCHEDULER_ROLE` can schedule a multiplier change 300 days out, but then immediately overwrite it to take effect next block with a completely different value, i.e. the originally announced window provides no actual guarantee, and integrators relying on the first `UIMultiplierUpdated` event have effectively zero warning before the real change lands.

Recommendation: Consider adding a minimum delay inside `ScaledToken._setUIMultiplier` , so any scheduled multiplier change — whether an initial schedule or an overwrite — must sit for at least that delay before becoming effective. This bounds the worst-case acceleration to a fixed reaction window instead of one block, during which integrators can observe the `UIMultiplierUpdated` event and act accordingly.

Mantle: Fixed in PR 4.

Cantina Managed: Fixed by PR 4 that adds a `MIN_MULTIPLIER_NOTICE=24hours` enforced within `_setUIMultiplier()` .

3.2 Informational

3.2.1 Incorrect comment states `ReentrancyGuard` uses transient storage

Severity: Informational.

Context: TokenIssuer.sol#L72.

DRAFT

Description: The comment about `ReentrancyGuard` is incorrect. OpenZeppelin `ReentrancyGuard` does not use `tstore`. `ReentrancyGuardTransient` is the variant that uses transient storage. The current `ReentrancyGuard` usage is acceptable, but the comment misstates the implementation behavior.

Recommendation: Switch to `ReentrancyGuardTransient` if transient storage is intentionally required.

Mantle: Fixed in PR 2.

Cantina Managed: Fix verified.

3.2.2 Unused `Executed` event remains in `TokenIssuer`

Severity: Informational.

Context: TokenIssuer.sol#L40.

Description: `TokenIssuer` declares the `Executed` event, but the event is never emitted. Keeping unused events in the contract interface creates unnecessary noise and can mislead integrators or reviewers into expecting an execution flow that does not exist.

Recommendation: Remove the unused `Executed` event from `TokenIssuer`.

Mantle: Fixed in PR 5.

Cantina Managed: Fix verified.

3.2.3 ERC-8056 is a Draft Standard that needs to be monitored for upcoming changes

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: This codebase implements ERC-8056, which is currently in Draft status, not Final. Draft ERCs can still change — function signatures, interface IDs, or semantics may be revised as a result of ongoing community review, including active discussion on Ethereum Magicians.

Since interface IDs are hardcoded in `supportsInterface`, a spec change post-deployment could leave the contract advertising conformance with an interface ID that no longer matches the final ERC.

Recommendation: Consider:

1. Pinning the exact ERC-8056 draft revision this implementation targets (e.g. in code comments or project docs).
2. Monitoring the Ethereum Magicians thread and ERC PR history for changes affecting interface IDs, signatures, or semantics before mainnet/integrator-facing deployment.
3. Re-evaluating readiness once the ERC advances toward Review/Last Call/Final to have an upgrade plan ready given the contracts are upgradeable.

Mantle: Acknowledged. We'll continue to monitor the ERC-8056 spec and are prepared to upgrade the contracts if the draft evolves in a way that affects interface IDs, signatures, or semantics — the beacon-proxy architecture is specifically in place to make that kind of upgrade tractable.

Cantina Managed: Acknowledged.

DRAFT

3.2.4 `DEFAULT_ADMIN_ROLE` lacks transfer safeguards

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: `TokenIssuer` and `ScaledToken` use `AccessControlEnumerableUpgradeable` for role management, with `DEFAULT_ADMIN_ROLE` controlling highly sensitive operations such as pausing, emergency withdraw sweeps, updating token details, compliance/pause managers and mint/burn enabling.

Under plain `AccessControlEnumerableUpgradeable`, admin transfer is a single, instant `grantRole` / `revokeRole` call with no built-in safeguards:

1. Multiple addresses can simultaneously hold `DEFAULT_ADMIN_ROLE`, i.e. nothing enforces a single accountable admin.
2. A transfer to the wrong address takes effect immediately and irreversibly, with no window to detect and cancel it.

Recommendation: Consider using `AccessControlDefaultAdminRulesUpgradeable` in place of (or alongside) `AccessControlEnumerableUpgradeable`. It enforces a single `DEFAULT_ADMIN_ROLE` holder at all times and requires a two-step, delayed handover, giving a window to catch and cancel malicious or mistaken admin transfers.

Mantle: We acknowledge the risk described. `TokenIssuer` and `ScaledToken` will continue to use AccessControlEnumerableUpgradeable; `DEFAULT_ADMIN_ROLE` is held by a multisig, not an EOA. The admin is a multisig with a relatively large signer set, and every transaction (including any role grant/revoke or admin handover) is fully reviewed by signers before execution. This provides the ”detect before it takes effect” window that the two-step handover is designed to add, at the process/governance layer rather than in-contract.

Multiple simultaneous admins is not a concern in our operating model. Admin authority is exercised through the single multisig; the enumerable extension is used deliberately for on-chain visibility of role members.

Cost/benefit. Adding the two-step delayed handover and single-admin invariant would not materially reduce residual risk given the multisig + review process, so we accept the risk as-is.

Note that we already harden role management beyond the OpenZeppelin defaults where it matters operationally:

`grantRole` disallows self-grants (`SelfGrantNotAllowed`), forcing the admin to delegate operational roles to separate accounts (`ScaledToken.sol:300-308`).

Cantina Managed: Acknowledged.

3.2.5 `receive()` is unnecessary in `TokenIssuer`

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: `TokenIssuer` declares `receive()externalpayable{}`, but none of its actual functionality needs native tokens — mint, burn, and transfer all operate on ERC-20 tokens, and no function is payable. The only reason native tokens could ever be routed to `receive()` is if someone mistakenly sends it there directly. The only mechanism to get it back out is the admin-gated `withdraw`, meaning any native tokens sent here sits idle and inaccessible to the sender until an admin discretionarily recovers it.

Recommendation: Consider removing `receive()` and the `token==address(0)` branch of `withdraw`.

DRAFT

Mantle: Fixed in PR 9.

Cantina Managed: Fix verified.

3.2.6 Missing zero-address check on `admin_` in `ScaledToken.__scaledToken_init`

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: `__scaledToken_init` grants `DEFAULT_ADMIN_ROLE` to `admin_` with no zero-address check, unlike `compliance_` / `pauseManager_`, which are independently validated inside `_setCompliance` / `_setPauseManager`. Currently `ScaledTokenFactory.deployToken` checks `_admin!=address(0)` before calling `initialize`, but that's caller-side only and the token has no defense-in-depth. If `initialize` is ever called outside that guarded path, `DEFAULT_ADMIN_ROLE` could be granted to `address(0)`, permanently locking the token out of all admin-gated functions with no recovery.

Recommendation: Consider adding zero-address check in `__scaledToken_init`, matching the pattern already used for the other initializer parameters.

Mantle: Fixed in PR 8.

Cantina Managed: Fix verified.

3.2.7 `SymbolMismatch` revert passes a description string as the provided value, not actual data

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

Description: `errorSymbolMismatch(stringexpected,stringprovided);` is meant to carry two comparable values — what was expected vs. what was actually provided. But `renameSymbol` calls it as: `revertSymbolMismatch(oldSymbol,"tokennotregisteredunderoldSymbol");` where provided isn't the actual value that caused the mismatch — it's a hardcoded string. This defeats the purpose of typed custom error parameters: any indexer, block explorer, or frontend that decodes `SymbolMismatch` and displays `provided` expecting a symbol/address will instead show the literal string `"tokennotregisteredunderoldSymbol"`, and there's no way to programmatically recover what was actually registered.

Recommendation: Consider passing the actual mismatched `provided` value.

Mantle: Fixed in PR 7.

Cantina Managed: Fix verified.

3.2.8 Centralization risk from privileged roles and flows

Severity: Informational.

Context: *(No context files were provided by the reviewer)*.

The protocol has several privileged roles across contracts. Every role can independently cause material harm to users through a single misconfiguration or compromise. Without appropriate timelocks, effective multisigs, role-separation, monitoring and incident response in place, private key compromises can lead to loss/lock of user funds or protocol malfunctioning.

Role	Worst-case single-action impact	Misuse scenario if compromised

DRAFT

`proxyAdmin` (beacon owner, `ScaledTokenFactory`)	Replace the implementation logic for **every deployed `ScaledToken` simultaneously**, no timelock, no per-token opt-out	Push malicious bytecode rewriting `mint` / `_update` to mint unlimited supply, drain all holder balances, or strip compliance checks across the entire product line in one tx
`DEFAULT_ADMIN_ROLE` (`ScaledTokenFactory`)	`createToken` (mint new tokens with arbitrary admin/issuers), repoint `defaultCompliance` / `defaultPauseManager` for all future tokens	Deploy a rogue token impersonating a real instrument (see `assetCode` issue), or default future tokens onto a permissive compliance/pause registry
`DEFAULT_ADMIN_ROLE` (`ScaledToken`, per token)	Swap `compliance` / `pauseManager` to arbitrary contracts (`updateCompliance` / `updatePauseManager`), grant `ISSUER_ROLE` / `MULTIPLIER_SCHEDULER_ROLE` to any address, toggle mint/burn	Repoint compliance to a stub that always reports "compliant," silently nullifying the blocklist/sanctions gate for every holder; grant `ISSUER_ROLE` to an attacker-controlled contract
`ISSUER_ROLE` (`ScaledToken`, held by `TokenIssuer`)	`mint(amount)` — uncapped, no per-call or cumulative limit beyond a boolean gate (`ScaledToken.sol`)	Mint arbitrary supply to `msg.sender` in a single transaction, instantly diluting/devaluing all holders
`MULTIPLIER_SCHEDULER_ROLE` (`ScaledToken`)	`setUIMultiplier` — move the displayed value 1e-9x–1e9x, overwrite/accelerate a scheduled change to next block (see separate `_beforeMultiplierUpdate` issue)	Announce a benign schedule far out, then overwrite it moments before execution with a drastically different multiplier, catching integrators off guard
`DEFAULT_ADMIN_ROLE` (`TokenIssuer`)	`withdraw` sweeps the issuer's **entire** ERC-20/native balance to caller (`TokenIssuer.sol`); manages recipient whitelist; grants `OPS_ROLE`	Sweep all inventory held by the issuer (customer-facing float) to an attacker address in one call
`OPS_ROLE` (`TokenIssuer`)	Routes `mint` / `burn` / `transfer` of any token the issuer holds `ISSUER_ROLE` on, to any approved recipient	Mint and immediately forward tokens to a whitelisted recipient the attacker controls, or burn a legitimate holder's pre-approved balance
`DEFAULT_ADMIN_ROLE` (`ComplianceRegistry` / `PauseRegistry`)	Grants `OPS_ROLE` to any address	One compromised key away from full freeze/blacklist power over every token using this registry
`OPS_ROLE` (`ComplianceRegistry`)	`blockUsers` / `sanctionUsers` — instant, no delay or on-chain dispute path	Freeze any individual holder's ability to transact, or mass-sanction addresses (e.g. competitors, whistleblowers) with no recourse
`OPS_ROLE` (`PauseRegistry`)	`pauseToken` / `pauseGlobally` — instant halt	Freeze a single token or **every token backed by this registry** at once, e.g. to trap funds mid-transfer or during a market event

Recommendation: Apply defense in depth across every privileged role listed above, rather than relying on the off-chain convention that admin keys "should" be multisigs:

- **Key custody.** Hold every `DEFAULT_ADMIN_ROLE` (on `ScaledTokenFactory`, each `ScaledToken`, `TokenIssuer`, `ComplianceRegistry`, and `PauseRegistry`) and the `proxyAdmin` /beacon-owner key behind a multi-sig (≥3-of-5 hardware wallets, geographically distributed signers) — none of these

are currently enforced to be anything other than a plain EOA.

- **Timelocks, tiered by blast radius.** Route admin actions through a `TimelockController`:
  - A minimum 48–72 hour delay on changes affecting user economics — `updateCompliance` / `updatePauseManager` and role grants on `ScaledToken`, `setDefaultCompliance` / `setDefaultPauseManager` on `ScaledTokenFactory`, and `MULTIPLIER_SCHEDULER_ROLE`'s `setUIMultiplier` (which currently has no minimum delay at all, per the separate `_beforeMultiplierUpdate` issue).
  - A longer delay (≥72 hours) specifically on `proxyAdmin` beacon upgrades, since a single upgrade rewrites the logic of every deployed `ScaledToken` at once.
- **Key separation.** Ensure no single address holds both `DEFAULT_ADMIN_ROLE` and `OPS_ROLE` / `ISSUER_ROLE` on the same contract, and that `proxyAdmin` is a distinct key from every `DEFAULT_ADMIN_ROLE` — the codebase's `SelfGrantNotAllowed` checks enforce this only at the point of granting, not as an ongoing invariant if keys are reused externally.
- **Least privilege and rotation.** Scope each key to the minimum role it needs (e.g. `TokenIssuer`'s `OPS_ROLE` operators should not also hold `ComplianceRegistry` / `PauseRegistry` `OPS_ROLE`) and rotate keys — admin and operational alike — on a defined schedule.
- **Real-time monitoring.** Alert automatically on: `RoleGranted` / `RoleRevoked` events, `TokenPaused` / `GlobalPaused` (`PauseRegistry`), `UsersBlocked` / `UsersSanctioned` / `SanctionsReplaced` (`ComplianceRegistry`), `UIMultiplierUpdated` / `UIMultiplierChangeOverwritten` (`ScaledToken`), `Minted` / `Withdrawn` (`TokenIssuer`), and any beacon `Upgraded` event — these are exactly the actions in the table above capable of unilateral, protocol-wide impact.
- **Incident playbooks.** Maintain a documented response plan per failure mode relevant to this protocol — compromised admin/ops key, `pauseGlobally` circuit-breaker trip, an unauthorized `mint` / `withdraw` from `TokenIssuer`, a malicious `MULTIPLIER_SCHEDULER_ROLE` overwrite, or a rogue beacon upgrade — specifying first responders, escalation chain, and user communication SLA for each.

Mantle: We appreciate this recommendation and will give it full consideration. We agree that the protection of privileged roles should not rest on the off-chain convention that admin keys "should" be multisigs, and we will address it across both the operational and technical layers:

- Key custody & separation. All `DEFAULT_ADMIN_ROLE` holders (`ScaledTokenFactory`, each `ScaledToken`, `TokenIssuer`, `ComplianceRegistry`, `PauseRegistry`) and the `proxyAdmin`/beacon-owner will be held behind a multisig with geographically distributed hardware-wallet signers, kept distinct from operational (`OPS_ROLE` / `ISSUER_ROLE`) keys and from each other. We acknowledge that `SelfGrantNotAllowed` enforces separation only at grant time, and will maintain key separation as an ongoing operational invariant.
- Least privilege & rotation. We will scope each key to the minimum role it requires and adopt a defined key-rotation schedule for both admin and operational keys.
- Timelocks. We will evaluate routing admin actions through a `TimelockController` with delays tiered by blast radius — a minimum delay on user-economics-affecting actions and a longer delay on beacon upgrades.
- Monitoring & incident response. We will set up real-time alerting on the enumerated events (role changes, pause/global-pause, compliance sanctions, multiplier updates/overwrites, mint/withdraw, and beacon upgrades) and maintain documented incident playbooks per failure mode, with first responders, escalation chain, and user-communication SLAs.

We will incorporate these into our deployment and operational procedures.

Cantina Managed: Acknowledged.

DRAFT